Vehicle Dynamics Modeling and Simulation using 20-sim

Tommaso Magelli Giorgio Giaretta

June 25, 2026

Abstract

This report details the development of a comprehensive 6 Degree-Of-Freedom (6-DOF) vehicle dynamics model using the Bond Graph methodology in the 20-sim environment. The model is designed to simulate both longitudinal and lateral dynamics, incorporating non-linear tire behavior through the Pacejka and a fully coupled 3D rigid body chassis.

1 Introduction

The objective of this project was to build a complete 6 Degree-Of-Freedom (6-DOF) vehicle simulation from scratch, utilizing the Bond Graph methodology within the 20-sim modeling environment.

We modeled our architecture from scratch to explicitly define and monitor the physical forces and velocities involved. By using Bond Graphs, the model tracks the exchange of power in the system: the product of Effort (forces and torques) and Flow (linear and angular velocities)—ensuring that energy continuity and the fundamental laws of physics are maintained across the entire system.

The core of the simulation is a 6-DOF rigid body based on the Newton-Euler equations, which captures the roll, pitch, and yaw dynamics in 3D space. This central chassis is coupled to the ground through independent suspension models and the Pacejka Formula, which handles the non-linear tire behavior.

During the development process, we addressed several practical simulation challenges, including the management of static equilibrium (applying suspension preload to support the vehicle's weight at startup) and resolving numerical singularities associated with zero-speed tire slip.

This report outlines the step-by-step construction of the model. The document is structured as follows:

- **Section 2 (System Architecture):** Provides a global view of the macro-blocks and how they connect to each other.
- **Section 3 (Component Modeling):** Dives deep into the physics and the Bond Graph structures of the Car Body, the Wheels, the Differential, and the Tire models. It also explains how we configured the 20-sim environment, and set up coordinate transformations to track the car's global trajectory.
- **Section 4 (Results and Experiments):** Showcases the plots and 3D trajectories of the car under different testing scenarios, proving the validity of our model.

Figure 1: Internal structure of the Differential block representing the Final Drive ratio.

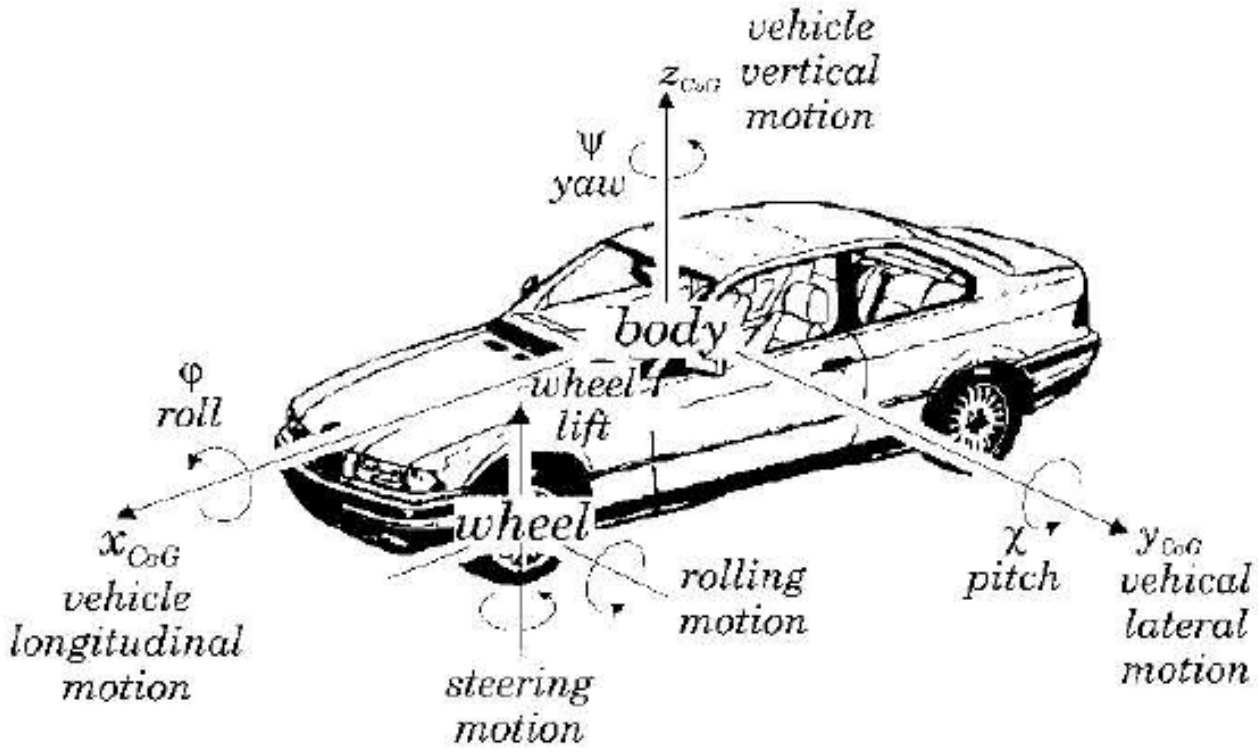


Figure 2: Conceptual representation of the vehicle's coordinate system and degrees of freedom.

2 System Architecture

The top-level architecture of the vehicle model acts as the blueprint of the entire simulation. Using the visual representation provided by 20-sim, the system is organized into macro-components that exchange energy and information. This modular approach allows each subsystem to be isolated, developed, and tested independently before being integrated into the full vehicle.

As shown in Figure 3, the architecture relies on two distinct types of connections:

- **Power Bonds (Half-arrows):** These connections transfer physical power between components. In the mechanical domain of this model, power is the product of an *Effort* variable (Forces in Newtons or Torques in Nm) and a *Flow* variable (Linear velocities in m/s or Angular velocities in rad/s).
- **Signal Bonds (Full arrows):** These connections transfer pure information without an energetic payload, such as a steering command or a target trajectory.

2.1 Macro-Components Overview

The model is composed of the following primary sub-systems, interconnected to simulate the complete vehicle environment:

1. **Powertrain Input:** The engine torque applied to the drivetrain is modeled using a Modulated Effort Source (`MSe`). To simulate a realistic driving scenario, the input command is governed by a piecewise-defined function, allowing the application of specific, time-varying torque profiles rather than abrupt steps.
2. **Path Controller:** A control block dedicated to feeding the model with a reference trajectory. Using a path-following control logic, it computes the necessary steering angle to keep the vehicle

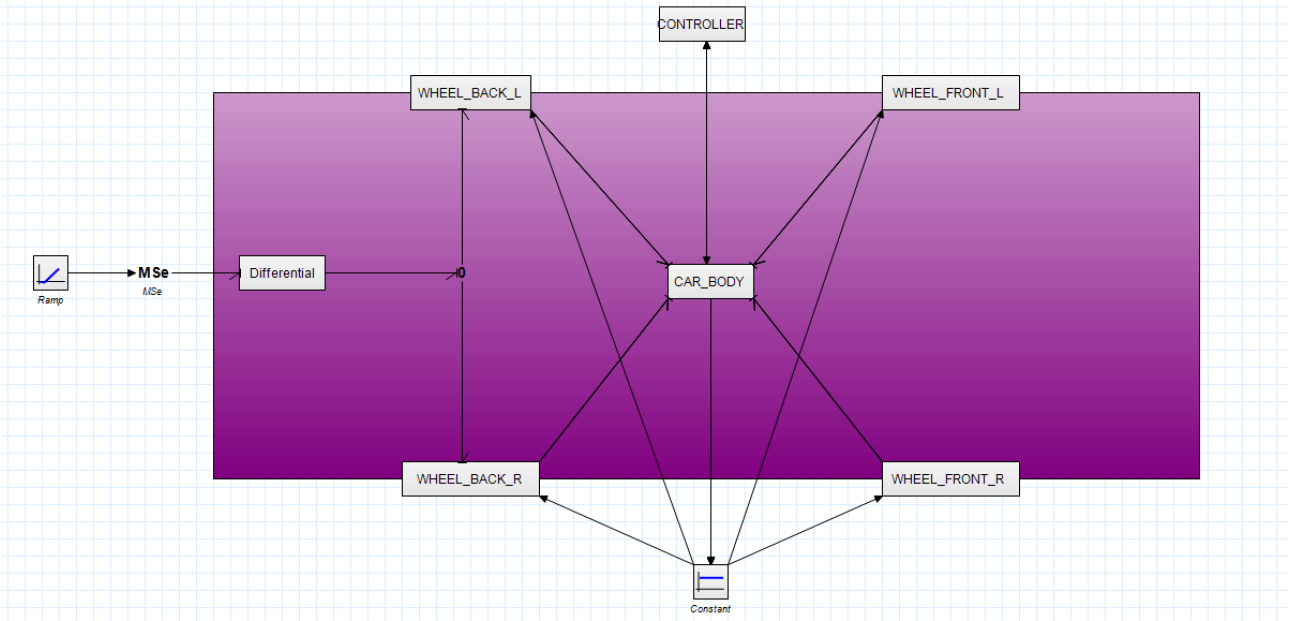


Figure 3: Top-level Bond Graph architecture of the vehicle model in 20-sim.

on the desired path and sends this information as a signal to the front wheels and the chassis.

3. **Road Surface:** An environmental block that defines the terrain profile. It computes and feeds the relative velocity between the tire contact patch and the ground. This block is essential for simulating road irregularities, such as bumps or potholes, and observing how the resulting energetic disturbances propagate through the wheels up to the car body.

4. **Differential:** This block receives the input torque from the `MSe` and splits it between the two rear driven wheels. It handles the kinematic relationship that allows the wheels to rotate at different speeds during cornering while maintaining the correct torque balance.

5. **Wheel Submodels (x4):** Each of the four wheels is encapsulated in its own submodel. These blocks contain the unsprung mass dynamics, the suspension elements (springs and dampers), and the tire-road interaction. Each wheel communicates with the central chassis through three parallel power bonds (representing the X, Y, and Z spatial dimensions), transmitting 3D forces and receiving 3D local velocities.

6. **Car Body (Chassis):** The `CAR_BODY` represents the sprung mass of the vehicle. It collects all the efforts (forces) generated by the tires and suspensions, integrates them through the Newton-Euler equations to determine the resulting accelerations, and updates the vehicle's position and orientation in space.

2.2 Causality and Physical Consistency

An advantage of the Bond Graph approach is the visualization of computational causality, indicated by the short perpendicular strokes at the ends of the power bonds. The causality strokes dictate which subsystem acts as the independent variable provider (imposing Effort or Flow) and which subsystem reacts.

The causality assignment in this model is not arbitrary; it strictly mirrors the physical nature of a mechanical mass-spring-damper system. In our top-level diagram, the causality strokes on the bonds connecting the wheels to the `CAR_BODY` are positioned adjacent to the `CAR_BODY` block.

This confirms that the chassis acts as a large multi-dimensional Inertia field I-element with integral causality. Physically, the `CAR_BODY` receives Efforts (forces) from the four suspensions, integrates

them over time to compute its internal momentum and velocity, and feeds the resulting Flows (local velocities) back to the wheels. Conversely, the suspension springs inside the wheel blocks act as Capacitances (C-elements), integrating those relative velocities to compute the resulting forces.

3 Component Modeling

3.1 The Car Body (6-DOF)

3.1.1 Why a 6-DOF Model?

Before diving into the equations governing the `CAR_BODY` block, it is essential to understand why a 6 Degree-Of-Freedom (6-DOF) model was chosen over simpler alternatives.

In mobile robotics and vehicle dynamics, the most common starting point is the Bicycle Model (a 2D model). The bicycle model simplifies the vehicle by collapsing the left and right wheels into a single track, restricting the motion to a flat, 2D plane. It only tracks three degrees of freedom: longitudinal velocity, lateral velocity, and yaw rate. While mathematically simple and computationally inexpensive—making it ideal for basic steering controllers and flat-ground path following—it completely ignores the vertical dimension.

However, a real vehicle is a 3D mass suspended on four springs. When a car accelerates, brakes, or takes a sharp corner, its inertia resists the change in motion, causing the chassis to tilt. This load transfer, compresses the suspensions unevenly, altering the normal vertical force (F_z) pushing each tire into the ground. Since the grip generated by a tire is non-linear and highly sensitive to this normal load, ignoring the 3D behavior means ignoring the true physical limits of the car. A 2D model simply cannot predict if a tire is about to lose traction due to unloading.

Transitioning to a full 3D car-like model introduces significant complexity. Instead of simple planar geometry, the 6-DOF model must track three translations (X, Y, Z) and three rotations (Roll, Pitch, Yaw). This requires the use of 3D rotation matrices to map tire forces into the chassis frame, Euler angle kinematics to track the global orientation, and full Newton-Euler equations that account for Coriolis effects. Despite the mathematical complexity, this 3D approach is the only way to accurately simulate the energetic interplay between the chassis, the suspensions, and the road surface.

The `CAR_BODY` block computes these dynamics through a structured, four-phase sequence: Force Accumulation, Dynamics, Kinematic Distribution, and Global Trajectory Tracking.

3.1.2 Force and Moment Accumulation

The first step within the `CAR_BODY` is to gather the efforts coming from the four wheels. Because the front wheels steer, the forces generated at their tire contact patches are not aligned with the longitudinal and lateral axes of the car body. We must apply a 2D rotation matrix based on the steering angle δ to project the front wheel forces into the body frame:

$$\begin{bmatrix} F_{x,b} \\ F_{y,b} \end{bmatrix}_{Front} = \begin{bmatrix} \cos\delta & -\sin\delta \\ \sin\delta & \cos\delta \end{bmatrix} \begin{bmatrix} F_{x,wheel} \\ F_{y,wheel} \end{bmatrix} \tag{1}$$

Once all forces are aligned with the chassis, we calculate the total forces and moments around the Center of Gravity (CG). The moments are calculated by multiplying the forces by their respective lever arms: a (CG to front axle), b (CG to rear axle), w (half-track width), and h (CG height). The moment equations directly reflect physical load transfer. For instance, a longitudinal accelerating force F_x applied below the CG generates a pitching moment proportional to $h \cdot F_x$.

```
// From Wheels to CG (Forces and Moments)

delta = delta_in;
```

```
// Front wheels account for steering delta
Fx_FL_b = x_car_body_FL.e * cos(delta) - y_car_body_FL.e * sin(delta);
Fy_FL_b = x_car_body_FL.e * sin(delta) + y_car_body_FL.e * cos(delta);

Fx_FR_b = x_car_body_FR.e * cos(delta) - y_car_body_FR.e * sin(delta);
Fy_FR_b = x_car_body_FR.e * sin(delta) + y_car_body_FR.e * cos(delta);

// Rear wheels are fixed (delta = 0)
Fx_BL_b = x_car_body_BL.e;
Fy_BL_b = y_car_body_BL.e;

Fx_BR_b = x_car_body_BR.e;
Fy_BR_b = y_car_body_BR.e;

// sum of forces on axes (Body Frame)
Fx_tot = Fx_FL_b + Fx_FR_b + Fx_BL_b + Fx_BR_b;
Fy_tot = Fy_FL_b + Fy_FR_b + Fy_BL_b + Fy_BR_b;

// Z-axis: Sum of vertical reactions minus gravity
Fz_tot = z_car_body_BL.e + z_car_body_BR.e + z_car_body_FL.e + z_car_body_FR.e
- m * 9.81;

// Torque calculation at CG
Mx_tot = w * (z_car_body_BL.e + z_car_body_FL.e - z_car_body_BR.e - z_car_body_FR.e)
+ h * Fy_tot;
My_tot = -a * (z_car_body_FL.e + z_car_body_FR.e) + b * (z_car_body_BL.e
+ z_car_body_BR.e) + h * Fx_tot;

// Yaw (Mz) uses individual forces ALREADY rotated to body frame
Mz_tot = a * (Fy_FL_b + Fy_FR_b) - b * (Fy_BL_b + Fy_BR_b)
- w * (Fx_FL_b - Fx_FR_b + Fx_BL_b - Fx_BR_b);
```

3.1.3 Newton-Euler Dynamics

With the total forces and moments available, the model calculates the dynamic behavior of the chassis. Because calculations are performed in the local reference frame of the car (which is moving and rotating), the Newton-Euler equations must include Coriolis and centrifugal terms. The resulting accelerations are defined as:

$$\begin{cases} \dot{v}_x = \frac{\sum F_x}{m} - \omega_y v_z + \omega_z v_y \\ \dot{v}_y = \frac{\sum F_y}{m} - \omega_z v_x + \omega_x v_z \\ \dot{v}_z = \frac{\sum F_z}{m} - \omega_x v_y + \omega_y v_x \end{cases} \tag{2}$$

$$\begin{cases} \dot{\omega}_x = \frac{\sum M_x + (J_y - J_z)\omega_y \omega_z}{J_x} \\ \dot{\omega}_y = \frac{\sum M_y + (J_z - J_x)\omega_z \omega_x}{J_y} \\ \dot{\omega}_z = \frac{\sum M_z + (J_x - J_y)\omega_x \omega_y}{J_z} \end{cases} \tag{3}$$

After calculating these accelerations, the 20-sim integration function `int()` computes the six state variables of the chassis.

```
// Newton-Euler Equations (At CG)

ax = (Fx_tot / m) - wy*vz + wz*vy;
ay = (Fy_tot / m) - wz*vx + wx*vz;
az = (Fz_tot / m) - wx*vy + wy*vx;

alphax = (Mx_tot + (Jy - Jz)*wy*wz) / Jx;
alphay = (My_tot + (Jz - Jx)*wz*wx) / Jy;
alphaz = (Mz_tot + (Jx - Jy)*wx*wy) / Jz;

vx = int(ax);
vy = int(ay);
vz = int(az);
wx = int(alphax);
wy = int(alphay);
wz = int(alphaz);
```

3.1.4 Kinematic Transformation

This phase answers a fundamental modeling question: Why do we need to calculate velocities backward, from the CG to the wheels?

Tires generate forces based on slip —the relative difference in velocity between the tire contact patch and the road. To accurately compute slip, the tire model (Pacejka) requires the exact velocity of its own specific wheel hub. Because the chassis is pitching, rolling, and yawing, a wheel located far from the CG experiences additional velocity components. For instance, when the car yaws (turns), the outer wheels travel faster than the inner wheels.

We use the rigid body velocity field equation to distribute the CG velocity to the four corners:

$$\vec{v}_{hub} = \vec{v}_{CG} + \vec{\omega} \times \vec{r}_{hub} \tag{4}$$

Moreover, the front wheels are steered. The Pacejka formula expects the longitudinal and lateral slip to be aligned with the tire's local orientation, not the car's body. Therefore, after finding the velocity of the front corners, we must apply an inverse rotation matrix $(-\delta)$ to rotate the velocity vector from the chassis frame into the individual tire frame:

$$\begin{bmatrix} v_{x,hub} \\ v_{y,hub} \end{bmatrix}_{TireFrame} = \begin{bmatrix} \cos\delta & \sin\delta \\ -\sin\delta & \cos\delta \end{bmatrix} \begin{bmatrix} v_{x,hub} \\ v_{y,hub} \end{bmatrix}_{BodyFrame} \tag{5}$$

```
// From CG to Wheels (Kinematic Transformation)

// --- Rear Wheels (Wheel Frame coincides with Body Frame) ---
x_car_body_BL.f = vx - wy*h - wz*w;
y_car_body_BL.f = vy + wx*h - wz*b;

x_car_body_BR.f = vx - wy*h + wz*w;
y_car_body_BR.f = vy + wx*h - wz*b;

// --- Front Wheels (Transformed to Wheel Frame for Pacejka) ---
v_FL_x_b = vx - wy*h - wz*w;
v_FL_y_b = vy + wx*h + wz*a;

// Output to FL wheel (Rotation by steering angle)
```

```
x_car_body_FL.f = v_FL_x_b * cos(delta) + v_FL_y_b * sin(delta);
y_car_body_FL.f = -v_FL_x_b * sin(delta) + v_FL_y_b * cos(delta);

v_FR_x_b = vx - wy*h + wz*w;
v_FR_y_b = vy + wx*h + wz*a;

// Output to FR wheel
x_car_body_FR.f = v_FR_x_b * cos(delta) + v_FR_y_b * sin(delta);
y_car_body_FR.f = -v_FR_x_b * sin(delta) + v_FR_y_b * cos(delta);

// --- Front Wheels Z
z_car_body_FL.f = vZ + phidot*w - thetadot*a;
z_car_body_FR.f = vZ - phidot*w - thetadot*a;

// --- Rear Wheels Z ---
z_car_body_BL.f = vZ + phidot*w + thetadot*b;
z_car_body_BR.f = vZ - phidot*w + thetadot*b;
```

3.1.5 Global Frame and Trajectory Tracking

The velocities computed internally are relative to the car's local axes. To plot the car on a map and provide accurate position data to the path-following controller, we must project these local states into the fixed Global World Frame using the Z-Y-X Euler angle sequence.

First, the local angular velocities (w_x, w_y, w_z) are mapped to Euler angle rates $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ via the inverse Euler kinematic matrix:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \frac{\sin\phi}{\cos\theta} & \frac{\cos\phi}{\cos\theta} \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \tag{6}$$

These rates are integrated to find the absolute orientation of the car. Then, a full 3D rotation matrix $R_{ZYX}(\phi, \theta, \psi)$ is applied to the local linear velocities to find the global velocities (v_X, v_Y, v_Z):

$$\begin{bmatrix} v_X \\ v_Y \\ v_Z \end{bmatrix}_{Global} = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}_{Local} \tag{7}$$

These global velocities are finally integrated to compute the vehicle's absolute coordinates (X, Y, Z), which are exported as signals to the control systems and visualization scopes.

```
// Trasformazione da Body a World (Eulero Z-Y-X)

// Calcolo delle derivate degli angoli di Eulero
phidot = wx + wy * sin(phi) * tan(theta) + wz * cos(phi) * tan(theta);
thetadot = wy * cos(phi) - wz * sin(phi);
psidot = (wy * sin(phi) + wz * cos(phi)) / cos(theta);

// Integrazione per ottenere l'orientamento globale del veicolo
phi = int(phidot);
theta = int(thetadot);
psi = int(psidot);

// Trasformazione delle velocità dal Body Frame al World Frame tramite
```

```
matrice di rotazione
vX = vx*cos(psi)*cos(theta) + vy*(cos(psi)*sin(theta)*sin(phi) - sin(psi)*cos(phi))
     + vz*(cos(psi)*sin(theta)*cos(phi) + sin(psi)*sin(phi));
vY = vx*sin(psi)*cos(theta) + vy*(sin(psi)*sin(theta)*sin(phi) + cos(psi)*cos(phi))
     + vz*(sin(psi)*sin(theta)*cos(phi) - cos(psi)*sin(phi));
vZ = -vx*sin(theta) + vy*cos(theta)*sin(phi) + vz*cos(theta)*cos(phi);

// Integrazione per ottenere la Posizione Globale (Traiettoria X, Y, Z)
X_global = int(vX);
Y_global = int(vY);
Z_global = int(vZ,h);

// Trasformazione delle forze totali dal Body Frame al World Frame
FX_global = Fx_tot*cos(psi)*cos(theta) + Fy_tot*(cos(psi)*sin(theta)*sin(phi)
- sin(psi)*cos(phi))
            + Fz_tot*(cos(psi)*sin(theta)*cos(phi) + sin(psi)*sin(phi));
FY_global = Fx_tot*sin(psi)*cos(theta) + Fy_tot*(sin(psi)*sin(theta)*sin(phi)
+ cos(psi)*cos(phi))
            + Fz_tot*(sin(psi)*sin(theta)*cos(phi) - cos(psi)*sin(phi));
FZ_global = -Fx_tot*sin(theta) + Fy_tot*cos(theta)*sin(phi) + Fz_tot*cos(theta)*cos(phi);
```

3.2 The Wheel Subsystem

The wheel is modeled as a rigid body with "decoupled" dynamics. It is physically reasonable to neglect the cross-coupling between the vertical, longitudinal, and lateral axes, as the angular velocity components about the vertical and horizontal longitudinal axes are extremely small. Furthermore, gyroscopic effects are deemed negligible for the scope of this simulation.

Each wheel subsystem in the 20-sim model is constructed using three independent Bond Graph networks, one for each spatial dimension (X, Y, Z). This decoupling ensures that integral causality exists for all sub-models, leading to stable state equations.

3.2.1 Vertical Dynamics (Z-Axis) and Suspension

The vertical submodel represents the classical suspension system. It captures the interaction between the road, the unsprung mass of the wheel assembly, and the sprung mass of the vehicle body.

The architecture relies on specific Bond Graph junctions:

- **Modulated Flow Source (MS_f):** Represents the vertical velocity of the road surface profile (e.g., bumps or potholes).
- **Bottom 0-Junction:** Computes the relative velocity between the road surface and the wheel hub. This relative velocity flows into the `C3` element, which represents the radial stiffness of the pneumatic tire (k_t), generating the normal contact force (F_z or N).
- **Central 1-Junction:** Represents the vertical velocity of the unsprung wheel mass (M_t), modeled by the Inertia element `I3`.
- **Top 0-Junction:** Computes the relative vertical velocity between the wheel hub and the Car Body attachment point. This relative flow is fed into the main suspension components: the spring (`C2`, stiffness k_s) and the damper (`R2`, damping b_s).

The resulting effort (suspension force) is then transmitted to the `CAR_BODY` via the `z_car_body` port.

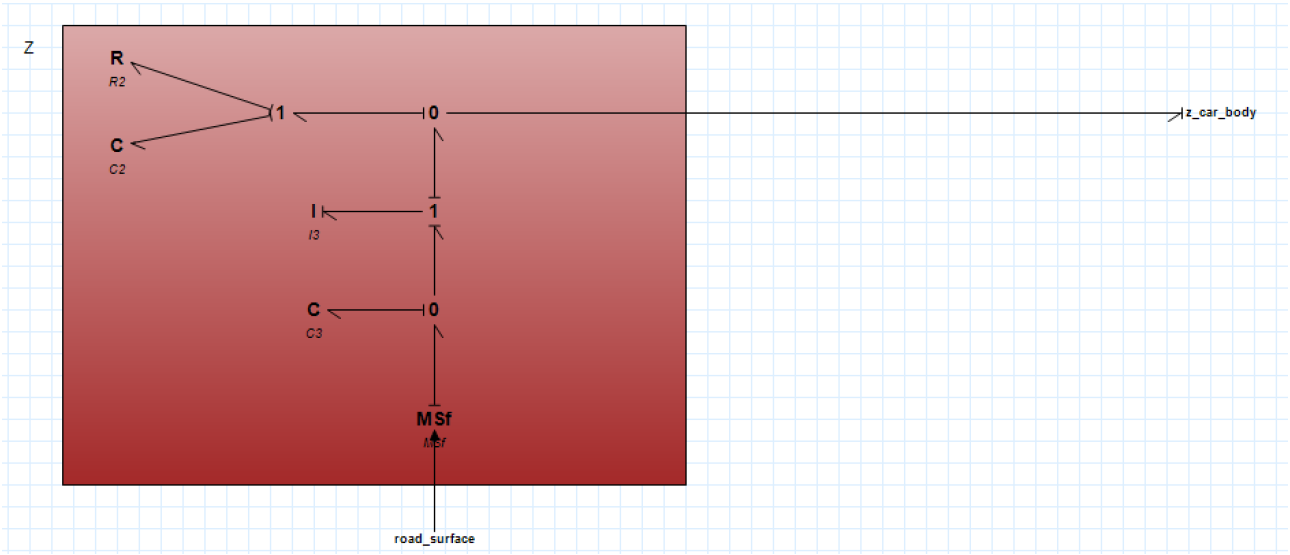


Figure 4: Bond Graph representation of the vertical wheel dynamics and suspension.

3.2.2 Longitudinal Dynamics (X-Axis)

The longitudinal Bond Graph is responsible for converting the engine torque into forward traction and modeling the longitudinal tire slip during acceleration and braking.

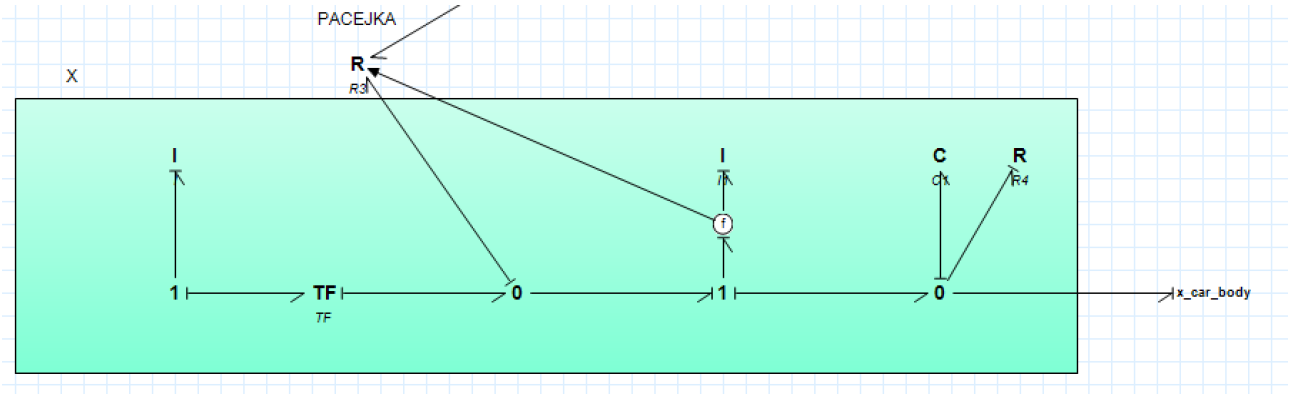


Figure 5: Bond Graph representation of the longitudinal wheel dynamics and drive torque.

The flow of power in this axis is as follows:

- **First 1-Junction (Left):** Represents the rotational velocity of the wheel (ω_t). The polar inertia of the wheel (I_t) is attached here. The input drive torque (τ) accelerates this inertia.
- **Transformer (TF):** Modulates the transformation from rotational to linear domain. The modulus of the transformer is the loaded tire radius (R), effectively computing the theoretical forward velocity of the tire tread ($R\omega_t$).
- **Central 0-Junction:** This is a critical node. It compares the tread velocity ($R\omega_t$) with the actual forward velocity of the wheel hub (u_t). The difference is the *slip velocity*, which flows into a modulated `R`-element (Pacejka model) to generate the longitudinal force F_L.
- **Second 1-Junction:** Represents the longitudinal linear velocity of the wheel mass (M_t).
- **Right 0-Junction:** Contains the longitudinal bushing stiffness (`C1`) and damping (`R4`), which filter the high-frequency tire vibrations before transmitting the final longitudinal force to the `CAR_BODY`.

3.2.3 Lateral Dynamics (Y-Axis)

When the vehicle steers, the absolute velocity of the wheel center is no longer perfectly aligned with the plane of the wheel. The tire tread deforms laterally, generating a side force known as the Cornering Force (F_C). The lateral Bond Graph models this behavior.

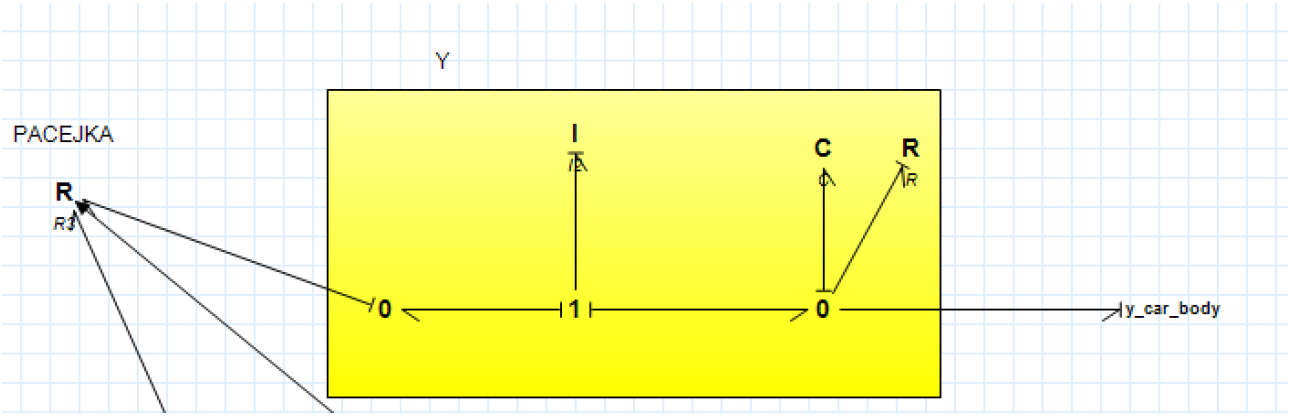


Figure 6: Bond Graph representation of the lateral wheel dynamics.

Unlike the longitudinal axis, there is no direct driving torque here. The lateral model operates as a passive reactive system:

- **Left 0-Junction:** Receives the lateral slip velocity. This node is connected to the lateral Pacejka `R`-element, which computes the Cornering Force based on the Slip Angle α (the angle between the absolute velocity vector and the tire plane).
- **Central 1-Junction:** Represents the lateral linear velocity of the wheel mass (M_t).
- **Right 0-Junction:** Similar to the longitudinal axis, it incorporates the lateral bushing stiffness (`C`) and damping (`R`) to simulate the compliance of the suspension arms before transmitting the effort to the `y_car_body` port.

3.2.4 The Pacejka Tire Model

Tires do not behave like rigid friction blocks; their grip depends on how much the rubber "slips" against the asphalt. To model this non-linear behavior accurately, our `R`-elements in the longitudinal and lateral Bond Graphs rely on the Pacejka Formula.

The code execution within the tire model follows a strict logical sequence to compute the resulting forces.

Step 1: Computing Slip (s) and Slip Angle (α)

The longitudinal slip s is the normalized difference between the theoretical tangential velocity of the tire ($R\omega_t$) and the actual forward velocity of the hub (u_t). The lateral slip angle α is the arctangent of the lateral velocity divided by the longitudinal velocity. One major computational challenge in vehicle simulation is the division by zero when the car is stationary ($u_t = 0$). To prevent numerical singularities and solve the zero-speed issue, an ϵ (epsilon) parameter is introduced using a max() function. This acts as a mathematical floor, ensuring the denominator never reaches absolute zero.

Step 2: Pure Pacejka Forces

Once the slip values are known, they are fed into the Magic Formula:

$$F(x) = D \sin\left(C \arctan\left(Bx - E(Bx - \arctan(Bx))\right)\right) \tag{8}$$

Where x is the slip (s or α), and B, C, D, E are empirical coefficients representing the tire's stiffness, shape, peak friction, and curvature. This formula generates the pure longitudinal force (F_{L0}) and pure lateral cornering force (F_{C0}).

Step 3: Combined Slip Phenomenon
A tire has a finite amount of grip, defined by the friction circle. If a car is accelerating heavily (using most of its grip longitudinally), its lateral cornering capability is drastically reduced. To model this *combined slip* trade-off without creating overly complex interdependent curves, we apply loss factors ($g_{L\alpha}$ and g_{Cs}) computed using cosine functions.

Step 4: Output Efforts
Finally, the pure forces are multiplied by the loss factors and the normal vertical load (F_z). In this specific snippet, a constant static load ($F_{z,static}$) is used to stabilize the initial launch of the simulation, ensuring the forces oppose the slippage correctly.

```
// CARICO NORMALE STATICO
F_z_static = (mass_wheel + 300.0) * 9.81;

//CALCOLO VELOCITÀ REALI
r_omega_t = p_x.f + ut_in;

// CALCOLO SLIP
// epsilon per evitare la singolarità a velocità zero
s = p_x.f / max(ut_in, max(r_omega_t, epsilon));
alpha = arctan(p_y.f / max(ut_in, epsilon));

//FORMULE DI PACEJKA
F_L0 = pacejka_D_long * sin(pacejka_C_long * arctan(pacejka_B_long * s
       - pacejka_E_long * (pacejka_B_long * s - arctan(pacejka_B_long * s))));
F_C0 = pacejka_D_lat * sin(pacejka_C_lat * arctan(pacejka_B_lat * alpha
       - pacejka_E_lat * (pacejka_B_lat * alpha - arctan(pacejka_B_lat * alpha))));

//FATTORI DI PERDITA COMBINATA --
g_Lalpha = cos(pacejka_C_long * arctan(pacejka_B_long * alpha));
g_Cs     = cos(pacejka_C_lat  * arctan(pacejka_B_lat  * s));

//USCITE DI SFORZO VERSO IL TELAIO
// La forza della gomma deve OPPORSI allo scivolamento (reazione vincolare).
p_x.e = F_L0 * g_Lalpha * F_z_static;
p_y.e = (F_C0 * g_Cs     * F_z_static);
```

3.3 The Powertrain and Differential Subsystem

The role of the powertrain is to deliver the engine's torque to the road surface. In a real vehicle, this involves a gearbox (to multiply torque) and a differential (to distribute torque between the driven wheels while allowing them to rotate at different speeds during a turn).

In our 20-sim model, this entire mechanical transmission is captured through the combination of a simple internal sub-model and a specific Bond Graph junction in the top-level architecture.

The internal structure of the `Differential` block (Figure 7) consists of just three elements: an input port, a Transformer (`TF`), and a `1`-junction output. The `TF` element models the Drive Ratio of the transmission. It mathematically scales the incoming engine torque (τ_{in}) up and scales the angular velocity down, simulating the mechanical advantage of the gearbox:

$$\tau_{axle} = \text{ratio} \cdot \tau_{engine} \tag{9}$$

However, the actual "differential" behavior—the splitting of the torque—does not happen inside this block, but rather immediately after it, in the top-level architecture (refer back to Figure 3).

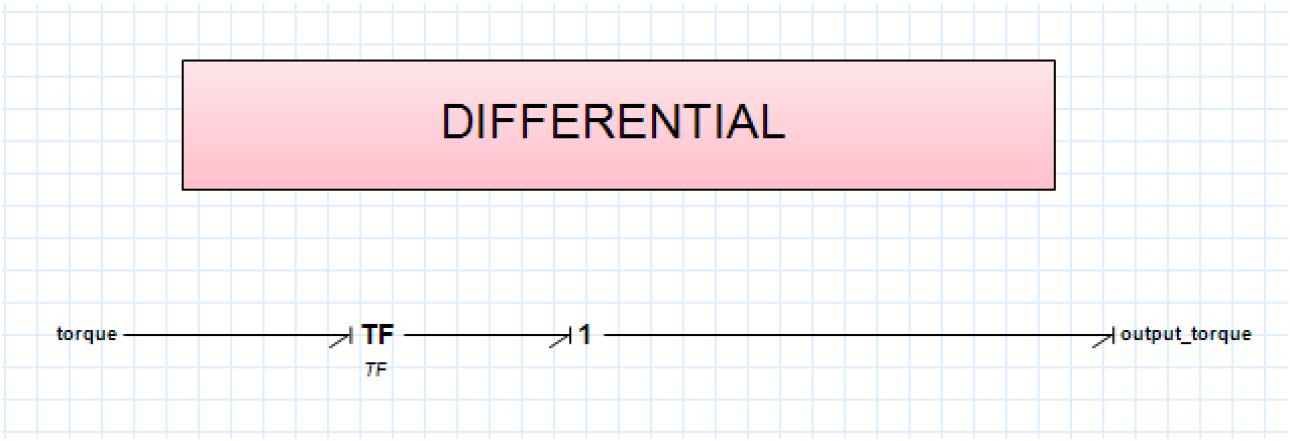


Figure 7: Internal structure of the Differential block representing the Final Drive ratio.

The output of the `Differential` block connects to a 0-Junction, which then branches out to the left and right rear wheels. In Bond Graph theory, a 0-junction represents a node where Effort is common across all connected bonds, and the Flows sum to zero. By connecting the drive axle to the two wheels via a 0-junction, the model perfectly enforces the laws of an open differential:

- **Equal Torque Distribution:** The torque is split exactly 50/50 between the left and right wheels ($\tau_{Left} = \tau_{Right} = \tau_{axle}$).
- **Kinematic Freedom:** The angular velocities are allowed to differ as long as their kinematic sum satisfies the input shaft speed, allowing the outer wheel to spin faster than the inner wheel during cornering without winding up the drivetrain.

3.4 The Path Controller Subsystem

While the mechanical components (chassis, wheels, drivetrain) handle the physical power flow of the vehicle, the `CONTROLLER` block operates entirely in the signal domain. It acts as the "virtual driver" of the simulation, guiding the vehicle along a predefined path and allowing us to test specific maneuvers, such as a smooth lane change or obstacle avoidance.

Because it does not exchange physical energy (Effort or Flow), it interfaces with the rest of the architecture using purely directional signal bonds (the full arrows in Figure 3).

Controller Interface

The controller creates a feedback loop with the `CAR_BODY`:

- **Inputs:** It continuously reads the vehicle's current state from the chassis block. Specifically, it tracks the global position (X, Y) and the absolute heading angle (Yaw, ψ).
- **Outputs:** Based on the path-tracking logic, it computes the required steering angle (δ_{req}) and feeds it back to the chassis, where it is used to rotate the reference frame of the front wheels.

Internal Logic Overview

The internal code of the controller is structured into three main logical steps:

1. **Target Generation:** The model projects a "lookahead" point slightly ahead of the vehicle. It then calculates a target lateral position (Y_{target}) using a smooth mathematical function. In this specific implementation, a scaled hyperbolic tangent function (`tanh`) is used to ensure a perfectly smooth transition from one lane to another, avoiding abrupt steering spikes.
2. **Error Evaluation:** The controller calculates the global error vector between the projected vehicle position and the mathematical target trajectory. Crucially, this global error is then

rotated by the vehicle's yaw angle (ψ) to transform it into a *local error* (y_{local})—representing the lateral distance to the target as "seen" directly from the nose of the car.

3. **Steering Actuation:** Using a geometric path-tracking approach, the controller determines the necessary path curvature required to eliminate the local lateral error. This required curvature is directly translated into a steering angle command δ using the vehicle's kinematic wheelbase (L).

```
// PUNTO DI LOOKAHEAD E TRAIETTORIA MATEMATICA
// Proiettiamo in avanti di Ld lungo la direzione attuale
X_target = X + Ld * cos(psi);

// Cambio di corsia: La funzione tanh passa da -1 a 1.
// Riscalata, fa passare Y da 0 a Y_amp in modo smooth.
Y_target = Y_amp * 0.5 * (1 + tanh(k_smooth * (X_target - X_start)));

// VETTORE DI ERRORE E TRASFORMAZIONE NEL LOCAL FRAME
dx = X_target - X;
dy = Y_target - Y;

// Ruotiamo il vettore errore usando l'angolo di imbardata (psi).
// A noi interessa solo l'errore laterale "visto" dal muso dell'auto.
y_local = -dx * sin(psi) + dy * cos(psi);


// GEOMETRIA DI CONTROLLO E STERZO RICHIESTO
// Calcolo della curvatura necessaria per beccare il bersaglio
curvature = (2.0 * y_local) / (Ld * Ld);

// Modello cinematico per l'angolo di sterzo richiesto alle ruote
delta_req = arctan(curvature * L);
```

3.5 The Road Surface Environment

Testing a vehicle model on a perfectly flat plane is sufficient to evaluate basic lateral handling and longitudinal acceleration. However, the true value of a full 6-DOF model with independent suspensions emerges only when environmental disturbances are introduced. To validate the pitch, bounce, and load transfer dynamics, we developed a `ROAD_SURFACE` block capable of generating terrain irregularities, such as a speed bump.

The physical profile of a road is a spatial function (Z elevation as a function of global X coordinate). However, the 20-sim dynamic simulation integrates over time (t). The `ROAD_SURFACE` block acts as a bridge between these domains.

It receives the current global X position of each individual wheel as an input signal from the `CAR_BODY`. By tracking where each tire is located on the map, the model determines whether the wheel is currently rolling over flat ground or over the speed bump.

Bump Generation

The bump is modeled as a smooth half-sine wave with a specific length and amplitude. If a wheel's X coordinate falls within the bump's range, its local Z position is calculated via the sine function.

Crucially, the output of this block cannot be a simple vertical position in meters. As seen in the vertical wheel Bond Graph (Figure 4), the road interfaces with the tire's radial stiffness via a Modulated Flow Source (`MSf`). Flow sources require a velocity input (m/s). Therefore, the code utilizes the `ddt()` function to compute the exact time derivative of the spatial elevation, converting the geometric bump into a continuous vertical velocity profile for each wheel.

```
// 1. Calcolo Posizione Z (Altezza del dosso spaziale)

// Ruota Anteriore Sinistra (FL)
if (in_X_FL >= start_bump) and (in_X_FL <= start_bump + bump_length) then
    z_pos_FL = bump_height * sin(pi * (in_X_FL - start_bump) / bump_length);
else
    z_pos_FL = 0.0;
end;
// (La stess logica è ripetuta per le ruote FR, BL, BR)

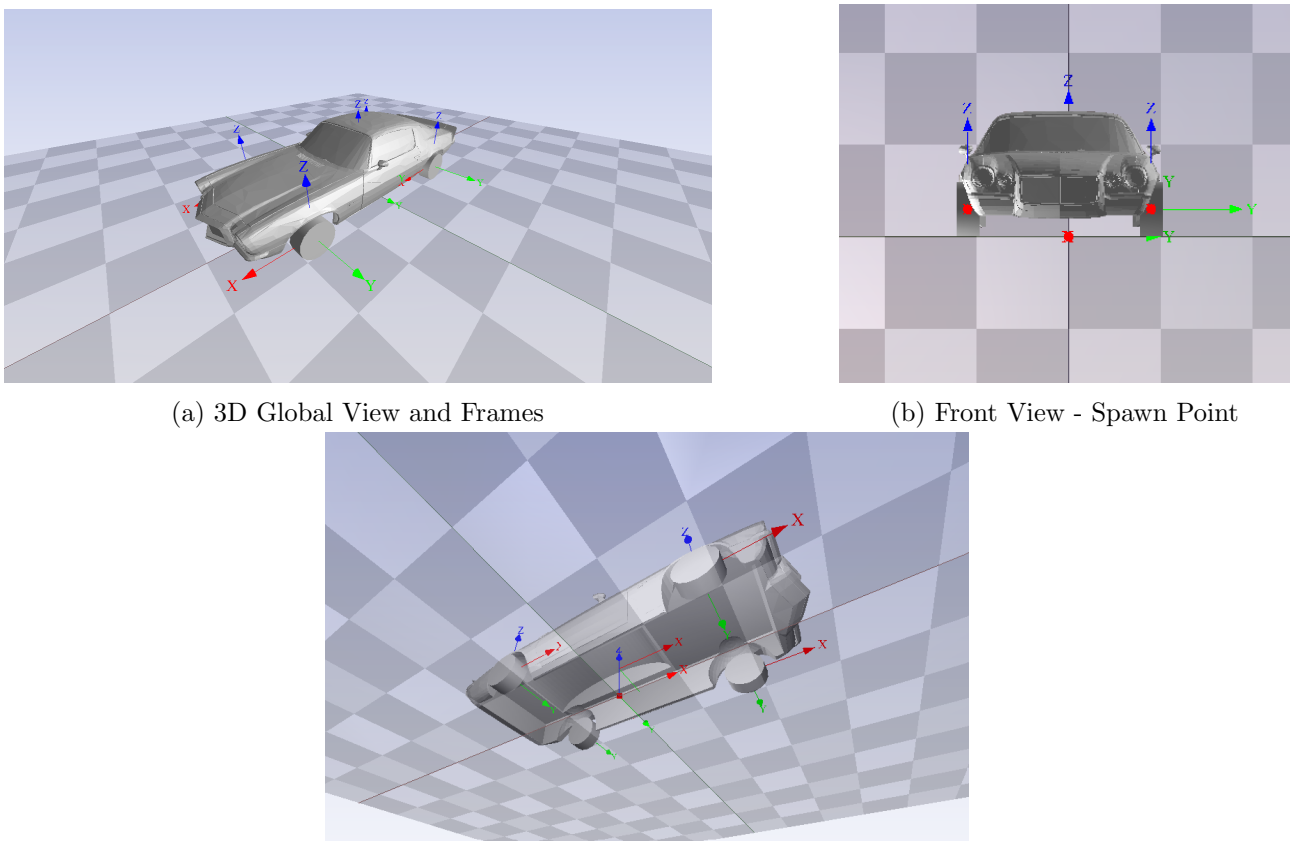
// 2. Derivazione nel tempo (Output verso i blocchi ruota)
// Converte l'altezza Z in velocità (Flusso) per alimentare le MSf
z_road_FL = ddt(z_pos_FL);
z_road_FR = ddt(z_pos_FR);
z_road_BL = ddt(z_pos_BL);
z_road_BR = ddt(z_pos_BR);
```

Expected Dynamic Response

By implementing this environment, we can expect a realistic chain of dynamic events. When the front tires hit the $X = 15.0$m mark, the `MSf` will inject a rapid upward velocity. This flow compresses the tire spring ($C3$), generating a spike in the normal force F_z. This force pushes the unsprung wheel mass upwards, compressing the main suspension spring ($C2$) and damper ($R2$), which in turn apply an upward thrust to the chassis, causing it to bounce and pitch backward. A moment later, the rear wheels will follow.

Most importantly, this violent fluctuation in the vertical load F_z will directly feed into the Pacejka model, temporarily altering the traction limits of the tires right as the vehicle traverses the bump.

4 Simulation Results and Analysis



(a) 3D Global View and Frames

(b) Front View - Spawn Point

(c) CG Frame Detail

Figure 8: Views of our vehicle in a 3d Environment

To validate the mathematical integrity of the 6-DOF model and the physical accuracy of the Bond Graph architecture, a series of simulation scenarios were executed. The scenarios are designed to progressively test the longitudinal, lateral, and vertical dynamics of the vehicle. To strengthen the data collection we designed a comprehensive simulation in a 3D environment to visualize the car actually move and respond to the inputs given.

4.1 Scenario 1: Longitudinal Acceleration (Flat Road)

In this initial test, the vehicle is placed on a perfectly flat road. A constant, positive engine torque is applied through the `MSe` block, while the steering angle is held at zero ($\delta = 0$). The primary objective is to verify the powertrain integration, the longitudinal tire slip (s) via the Pacejka model, and the Newton-Euler integration of the forward velocity (v_x).

As depicted in Figure 9 (Front Speed), the vehicle exhibits a smooth and continuous acceleration profile from a standstill. Driven by the input torque, the differential successfully distributes power to the rear wheels, and the non-linear Pacejka tire model accurately calculates the necessary longitudinal slip to generate forward traction. The vehicle reaches a velocity of approximately 8 m/s over the 10-second simulation window. Concurrently, the Global Position plot confirms a perfectly straight trajectory along the X-axis (reaching approximately 35 meters), validating that no spurious lateral forces or yaw moments are generated in the absence of steering input.

Beyond simple kinematics, this scenario perfectly demonstrates the 3D dynamic capabilities of the model, specifically regarding *longitudinal load transfer*. As the vehicle accelerates, the inertial resistance of the chassis creates a moment around the center of gravity. As seen in the Euler Angles

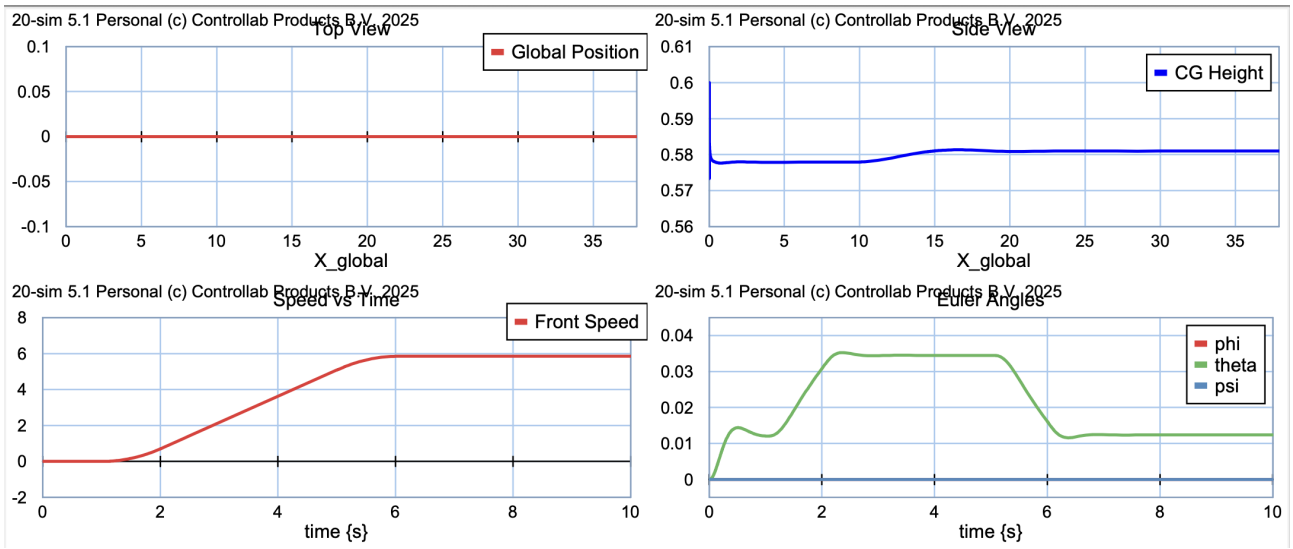


Figure 9: Global Position, Global Height, Longitudinal velocity (V_x), Euler angles, response under constant throttle input.

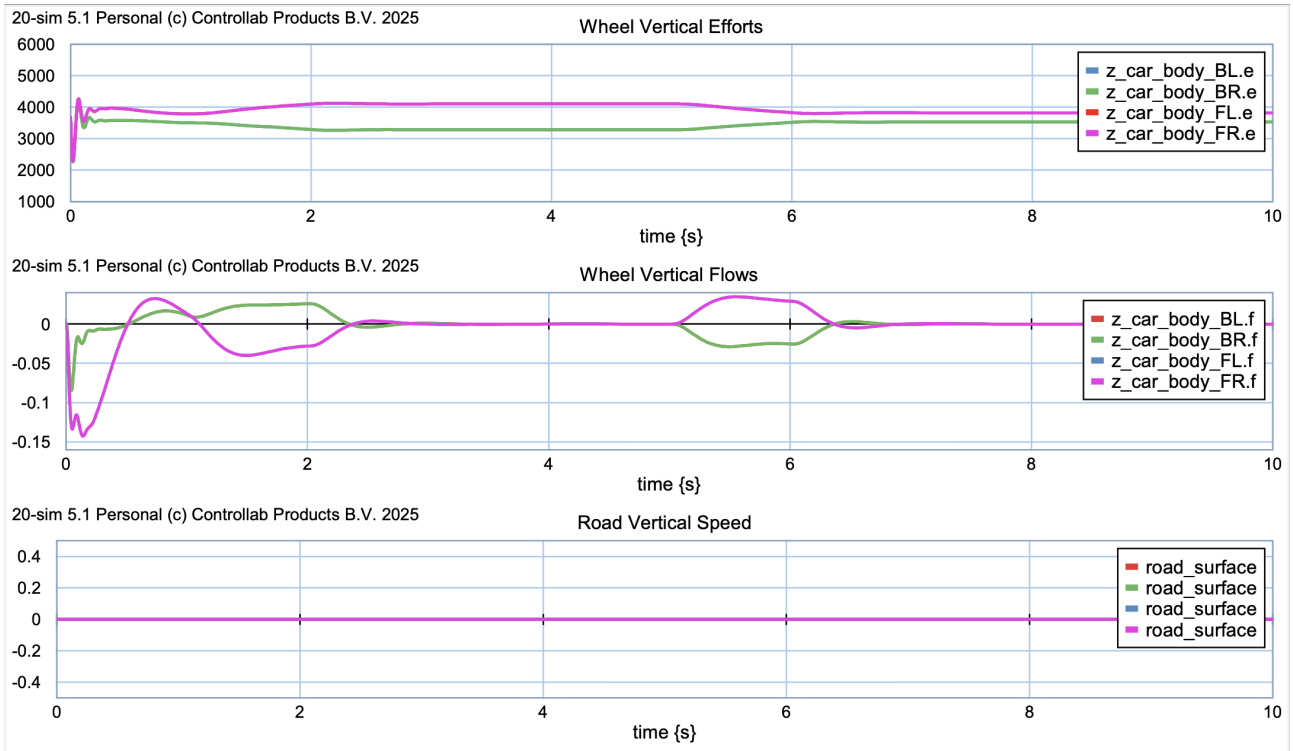


Figure 10: Chassis vertical tire load transfer during longitudinal acceleration.

plot (Figure 10), this causes the chassis to pitch backwards, indicated by an increase in the pitch angle (θ) up to approximately 0.04 rad.

This pitching motion directly affects the suspension springs. Analyzing the Wheel Vertical Efforts, a clear divergence from the static equilibrium load (approximately 3678 N per wheel) is observed. The acceleration causes the rear suspensions to compress, increasing the vertical normal force (F_z) on the rear tires, while the front suspensions extend, decreasing the normal force on the front tires. When the acceleration phase stabilizes, the pitch angle and the vertical efforts smoothly return towards their steady-state values. This accurately mirrors the "squat" phenomenon observed in real vehicles under hard acceleration and demonstrates that the energetic coupling between the longitudinal and vertical domains is functioning correctly.

4.2 Scenario 2: Lateral Dynamics and Path Following (Flat Road)

To evaluate the lateral dynamics and the effectiveness of the Path Controller, the vehicle is commanded to perform a smooth lane-change maneuver on a flat surface. This scenario validates the calculation of the tire slip angle (α), the generation of lateral cornering forces (F_C), and the resulting chassis yaw (ψ) and roll (ϕ) due to lateral load transfer.

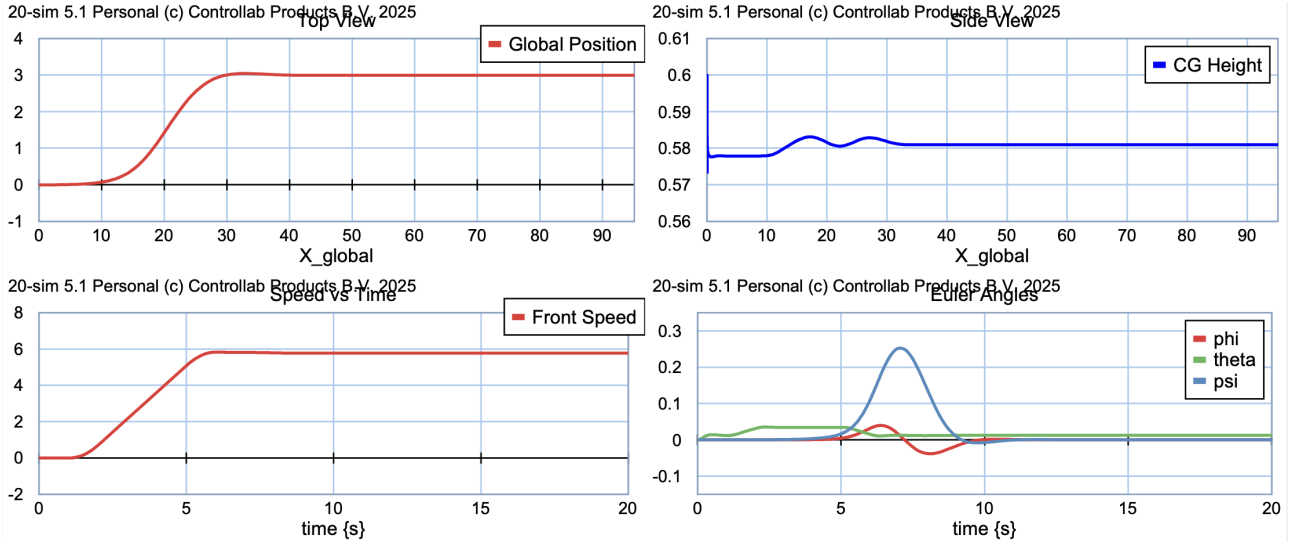


Figure 11: Top-down trajectory, Speed, Height, Angles ($X - Y$) of the vehicle executing a 3-meter lane change maneuver.

As shown in the Global Position plot (Figure 11), the Path Controller successfully guides the vehicle through a smooth 3-meter lateral shift. The vehicle accelerates to a steady cruise speed of approximately 5.8 m/s before initiating the maneuver around $t = 5s$.

The most significant validation of the 3D model lies in the chassis attitude and suspension response. Figure 12 (Euler Angles) illustrates a clear yaw motion (ψ) peaking at 0.25 rad to follow the path. Accompanying this yaw is a distinct, wave-like roll angle (ϕ). As the vehicle turns left into the new lane, lateral acceleration generates a centrifugal force acting on the CG, causing the chassis to lean towards the outside of the curve (positive roll, approximately 0.04 rad).

This lateral load transfer is captured in the Wheel Vertical Efforts plot (Figure 12). During the initial turning phase, the vertical forces acting on the four wheels diverge significantly. As the chassis rolls to the right, the right-side wheels (outer) experience a substantial increase in normal load, while the left-side wheels (inner) are correspondingly unloaded. Subsequently, when the vehicle counter-steers to align with the straight path in the new lane, the roll angle and the vertical efforts perfectly invert, loading the left wheels.

This dynamic distribution of the F_z forces is a critical physical phenomenon. Since it directly

alters the maximum grip limits of the individual tires within the Pacejka equations, it definitively proves the necessity and superiority of this 6-DOF approach over a simplified 2D planar model.

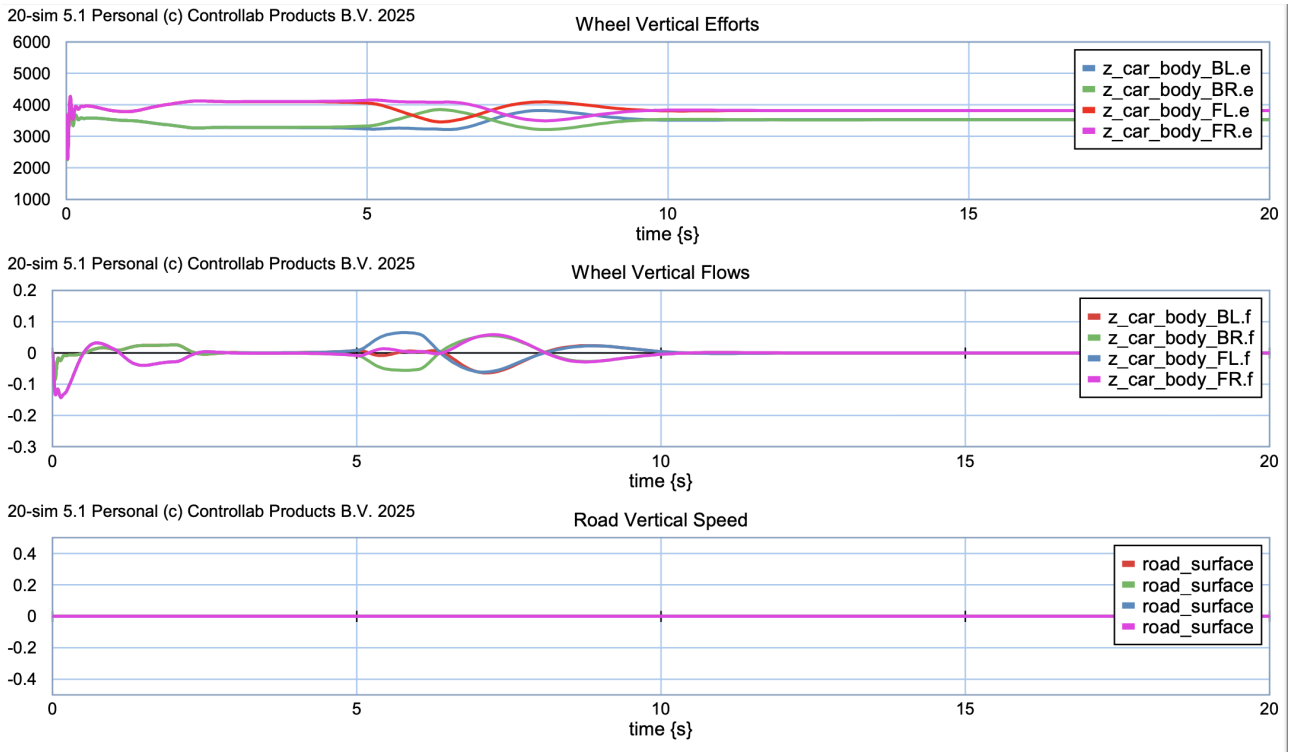


Figure 12: Lateral load transfer affecting the vertical tire forces (F_z) during the lane change.

4.3 Scenario 3: 3D Suspension Dynamics and Environmental Interaction (Speed Bump)

The final scenario serves as the ultimate test for the fully coupled 6-DOF architecture. The vehicle travels in a straight line but encounters the speed bump defined in the `ROAD_SURFACE` block. This test demonstrates the dynamic interaction between the road, the unsprung mass, the suspension elements, and the pitch dynamics of the sprung chassis.

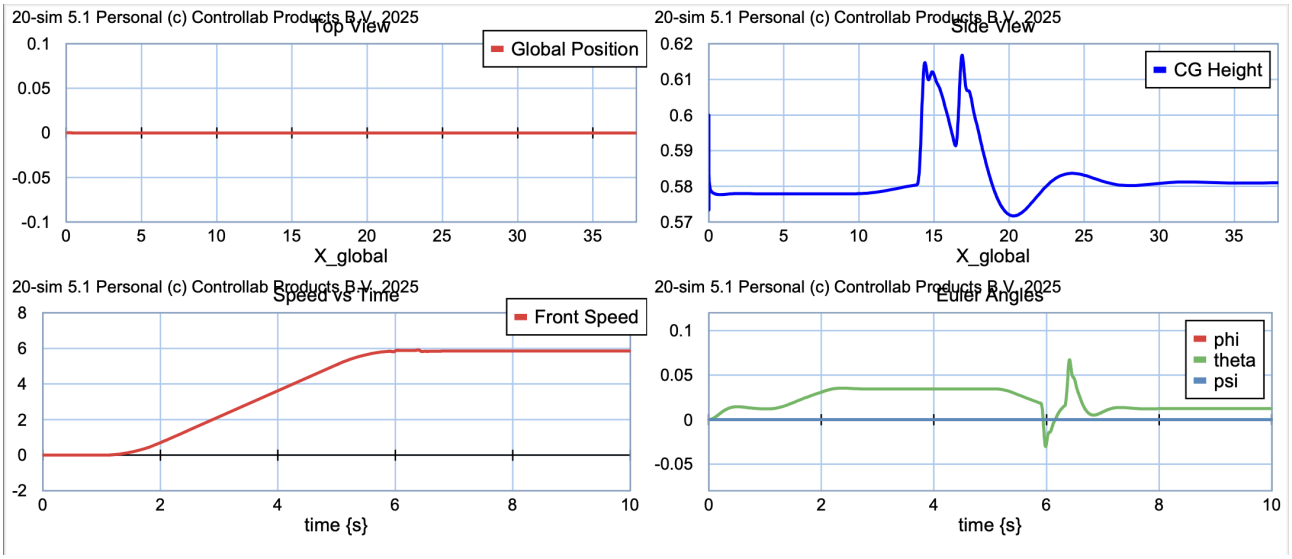


Figure 13: Chassis bounce (Z) and pitch angle (θ) variation due to the front and rear axle impacts.

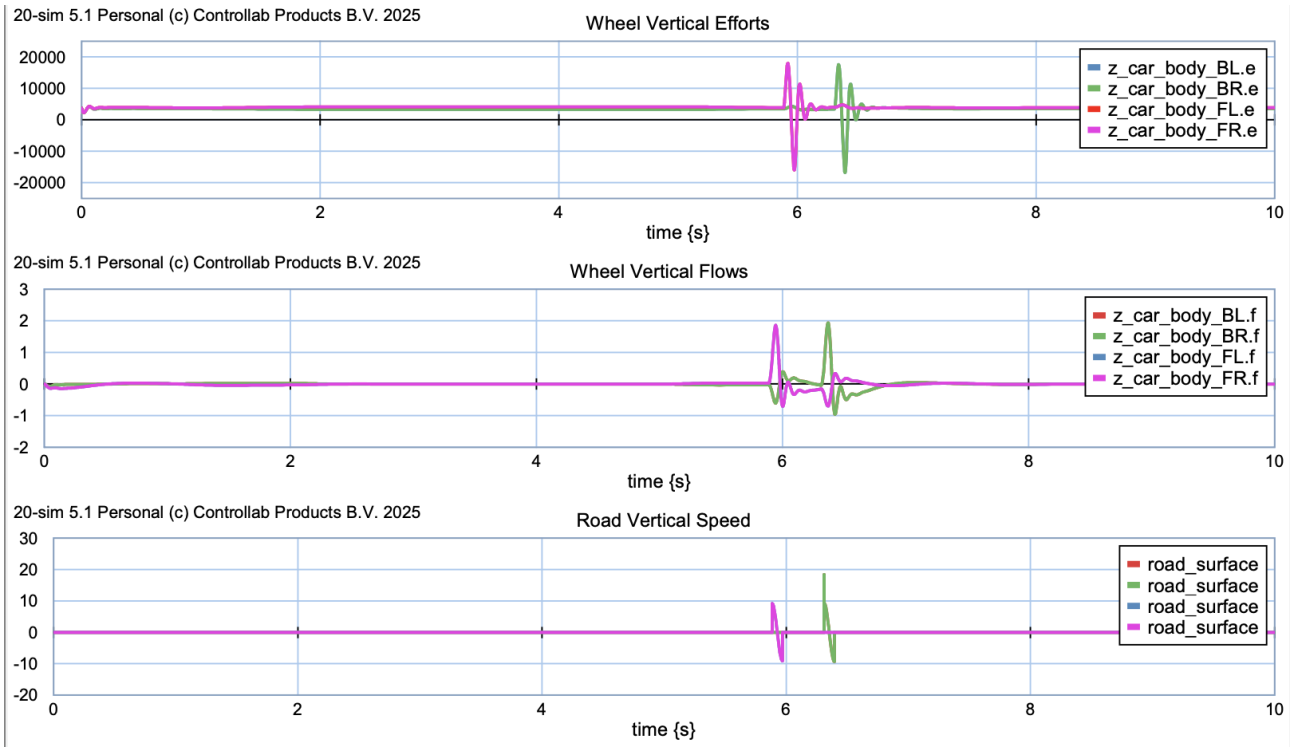


Figure 14: Dynamic variation of vertical tire loads (F_z) induced by the road surface disturbance.

As depicted in the Road Vertical Speed plot (Figure 14), the `ROAD_SURFACE` block correctly identifies the spatial position of the speed bump ($X = 15.0$ m) and injects a sharp, half-sine velocity pulse into the front wheels at approximately $t = 6s$. This is followed by an identical velocity pulse into the rear wheels after a short time delay, perfectly corresponding to the wheelbase distance and the vehicle's longitudinal speed.

This kinematic disturbance triggers a violent dynamic response in the suspension system. The Wheel Vertical Efforts plot illustrates a massive spike in the normal forces (F_z), momentarily exceeding 20,000 N on the front axle, followed by a sudden unloading phase (where forces drop significantly below the static weight, approaching negative values) as the vehicle crests the bump. The rear axle experiences the exact same phenomenon shortly after. This extreme fluctuation in normal load is crucial for evaluating tire traction limits, as Pacejka's lateral and longitudinal grip capabilities would be drastically reduced during the unloading phase.

The effect on the sprung mass (the chassis) is clearly visible in Figure 13. The CG Height (Side View) plot shows the center of gravity being thrown upwards from its 0.58 m steady-state elevation, clearing the 0.08 m obstacle with a complex double-peak signature representing the front and rear impacts. Concurrently, the Euler Angles plot captures the violent pitch (θ) oscillation, showing consecutive positive and negative peaks corresponding to the front and rear axles traversing the obstacle.

The suspension dampers (R-elements) perform their intended function beautifully: after the secondary impact from the rear wheels, the kinetic energy is rapidly dissipated, and the chassis height and pitch angle seamlessly return to a stable equilibrium without entering an endless resonant bouncing cycle.

5 Path Tracking Control Strategy: The Pure Pursuit

While Section 3.4 briefly introduced the `CONTROLLER` block's interfaces, this section delves into the theoretical framework governing the path-following behavior: the Pure Pursuit algorithm.

5.1 Trajectory Definition

In vehicle simulation, a target trajectory can be defined in two primary ways:

- **Discrete Waypoints:** A list of X, Y coordinates mapped on a grid. While common in GPS-guided systems, raw waypoints require complex interpolation algorithms to prevent abrupt, jagged steering commands.
- **Continuous Functions:** Mathematical equations defining Y as a function of X (or time).

For this project, we opted for a continuous mathematical function to guarantee infinite smoothness. The lane change maneuver was defined using a scaled hyperbolic tangent function (`tanh`). Unlike a sudden step-change or a sharp geometric angle, the `tanh` profile provides a gradual, continuous transition. This is crucial in vehicle dynamics: a smooth positional trajectory ensures continuous derivatives, meaning the vehicle will not experience impossible spikes in lateral acceleration or infinite jerk, allowing the Pacejka tire model to handle the load transfer realistically.

5.2 The Pure Pursuit Geometric Approach

The Pure Pursuit algorithm is widely used in robotics and autonomous driving because it mimics human driving behavior. When driving, a human does not look at the road directly immediately in front of the bumper; rather, they fixate on a target point on the path at a certain distance ahead.

The algorithm operates on a simple geometric principle:

1. **Lookahead Distance (L_d):** The algorithm projects a virtual point at a fixed distance L_d ahead of the vehicle's current position.
2. **Target Acquisition:** It finds the exact coordinates where this lookahead radius intersects the desired trajectory.
3. **Local Error Transformation:** As implemented in our 20-sim code, the global positional error between the vehicle and the target is rotated by the vehicle's yaw angle (ψ). This yields the *local lateral error* (y_{local})—representing exactly how far to the left or right the target is, relative to the car's nose.
4. **Arc Fitting:** The algorithm calculates the curvature (κ) of a circular arc that perfectly connects the vehicle's rear axle to the target point:

$$\kappa = \frac{2y_{local}}{L_d^2} \tag{10}$$

5. **Steering Angle:** Using a standard kinematic bicycle model, the calculated curvature is converted into the required steering angle (δ) at the front wheels, based on the vehicle's wheelbase (L):

$$\delta = \arctan(\kappa \cdot L) \tag{11}$$

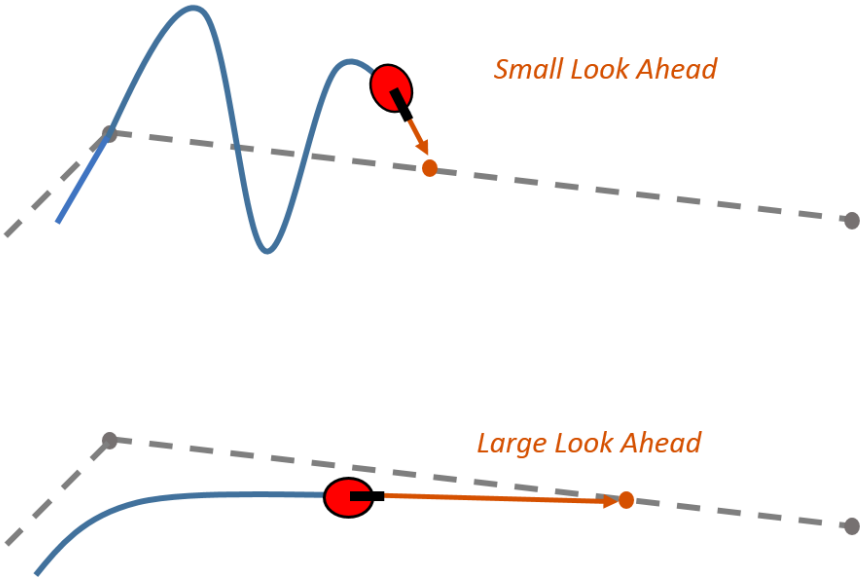


Figure 15: Geometric representation of the Pure Pursuit path tracking algorithm.

5.3 Controller Tuning: The Impact of Lookahead Distance

In our implementation, the critical tuning parameter is the lookahead distance (L_d). Its value profoundly impacts the vehicle's stability, and it must be carefully chosen in relation to both the vehicle's wheelbase (L) and its forward velocity.

- **If L_d is too short (Aggressive):** The target point is very close to the front bumper. The controller will demand sharp, violent steering angles to correct even minor lateral errors. If L_d approaches the length of the wheelbase ($L_d \approx L$), the geometry breaks down, causing severe oscillations. At high speeds, a short L_d guarantees instability, as the vehicle will violently overcorrect and potentially spin out due to Pacejka grip saturation.
- **If L_d is too long (Lazy):** The target point is far over the horizon. The controller will compute a very flat, gentle arc. While extremely stable, the vehicle will "cut corners" and fail to track tight curves accurately, dragging itself sluggishly toward the path.

Because of this extreme sensitivity, in real-world autonomous applications, L_d is rarely static; it is dynamically defined as a linear function of the vehicle's velocity ($L_d = k \cdot v_x$). However, for the specific 5.8 m/s cruise speed of our lane-change scenario, a fixed $L_d = 5.0$ m proved to be the optimal "sweet spot", providing a snappy yet damped response without inducing oscillations.

6 Conclusions

This project successfully demonstrated the construction of a 6-DOF vehicle dynamics model from the ground up using Bond Graphs in 20-sim. By avoiding pre-packaged black-box solutions, we achieved a transparent, energetically consistent simulation.

The modular architecture allowed us to independently verify the 3D Newton-Euler chassis dynamics, the non-linear Pacejka tire slip, and the multi-domain power transmission. The simulation scenarios confirmed that the model accurately captures critical real-world phenomena, including longitudinal squat during acceleration, lateral load transfer during cornering, and complex pitch and bounce interactions over environmental disturbances. Finally, integrating a geometric Pure Pursuit controller proved that the mechanical model is stable and responsive enough to handle closed-loop autonomous path tracking.